AWS Production-Ready Web Infrastructure using Terraform

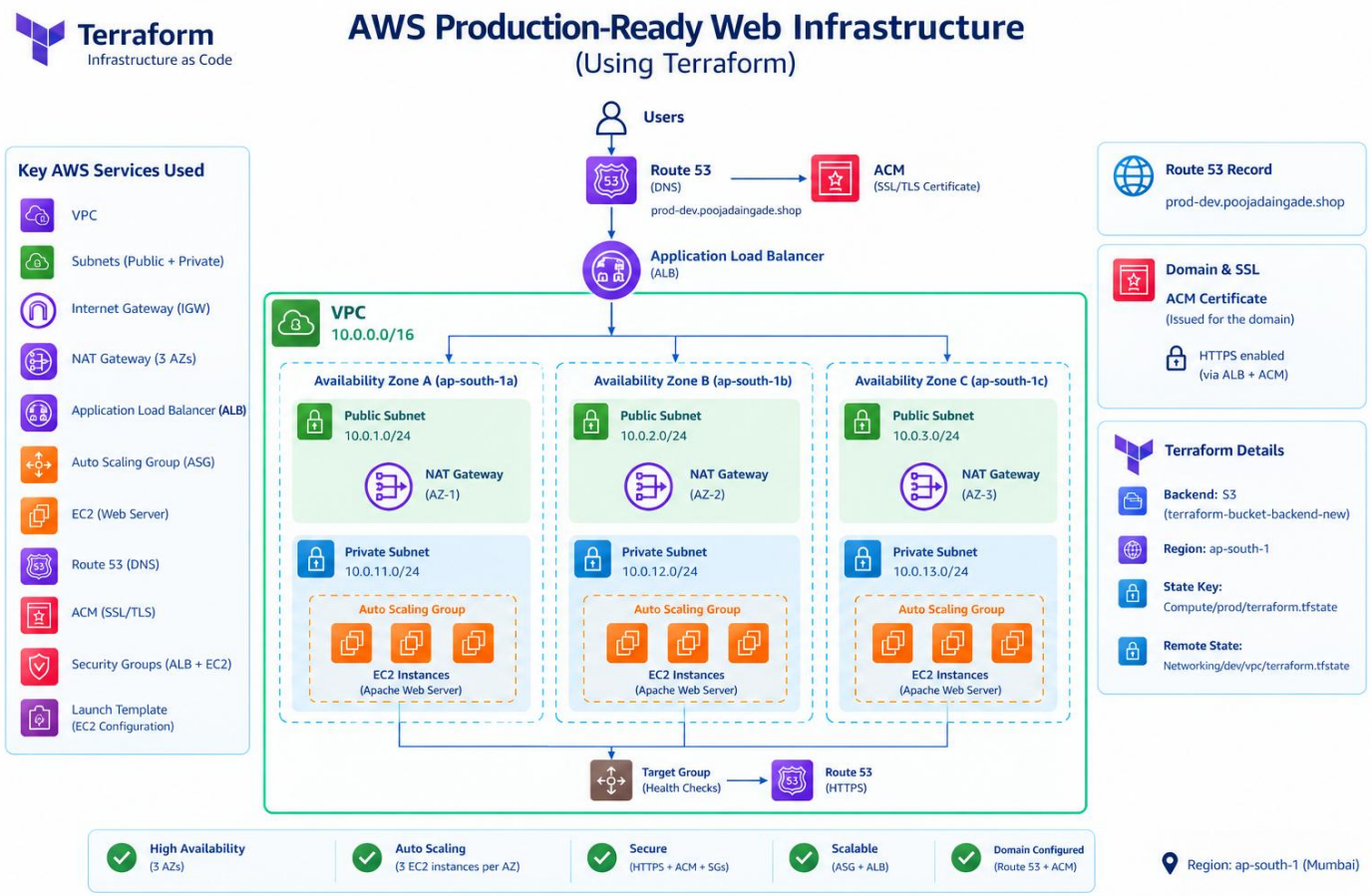
Prepared by: Pooja Daingade

Project Overview

The AWS Production-Ready Web Infrastructure using Terraform project focuses on provisioning and managing AWS infrastructure using Terraform (Infrastructure as Code). The project includes a VPC with public and private subnets, Internet Gateway, NAT Gateways, EC2 instances, Launch Template, Auto Scaling Group (ASG), Application Load Balancer (ALB), Target Group, AWS Certificate Manager (ACM), and Route 53 for domain configuration. Amazon S3 is used for Terraform remote state management.

The main objective of this project is to gain hands-on experience in AWS infrastructure provisioning, networking, load balancing, auto scaling, secure HTTPS configuration, and infrastructure automation using Terraform.

Architecture

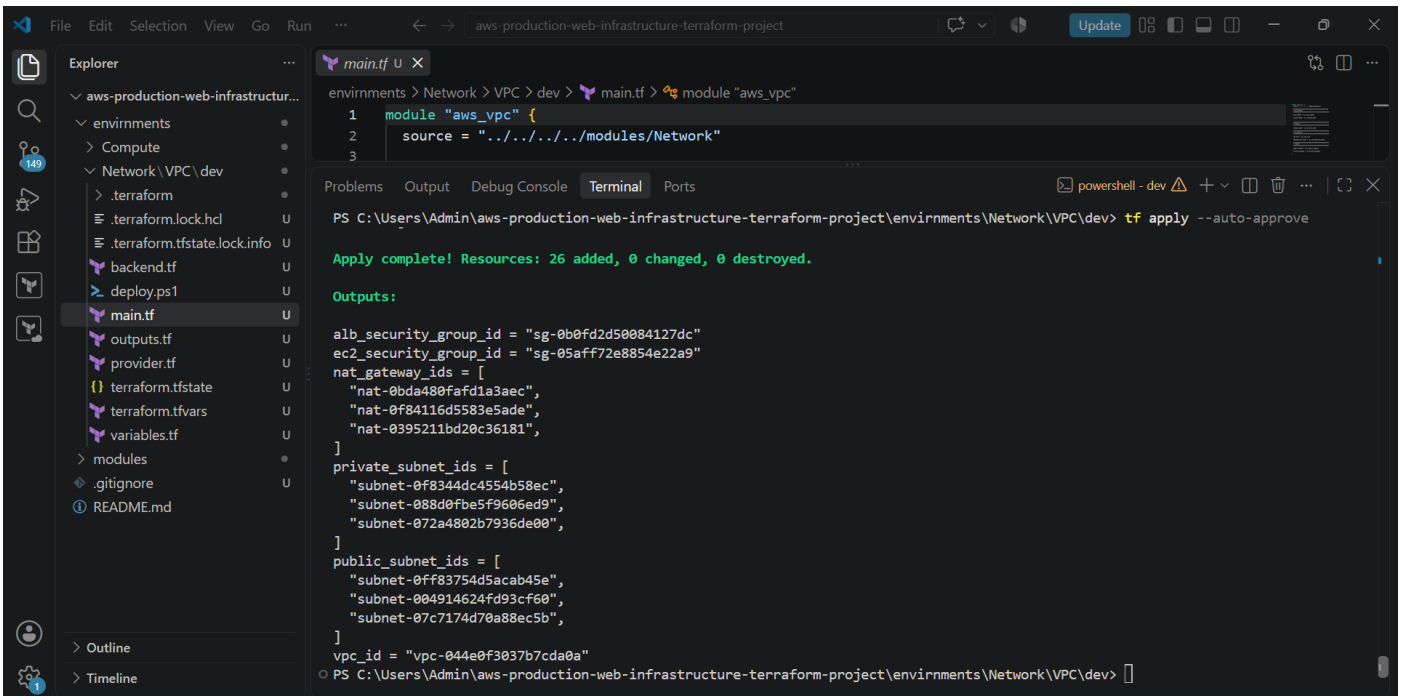


1. **VPC Configuration using Terraform**

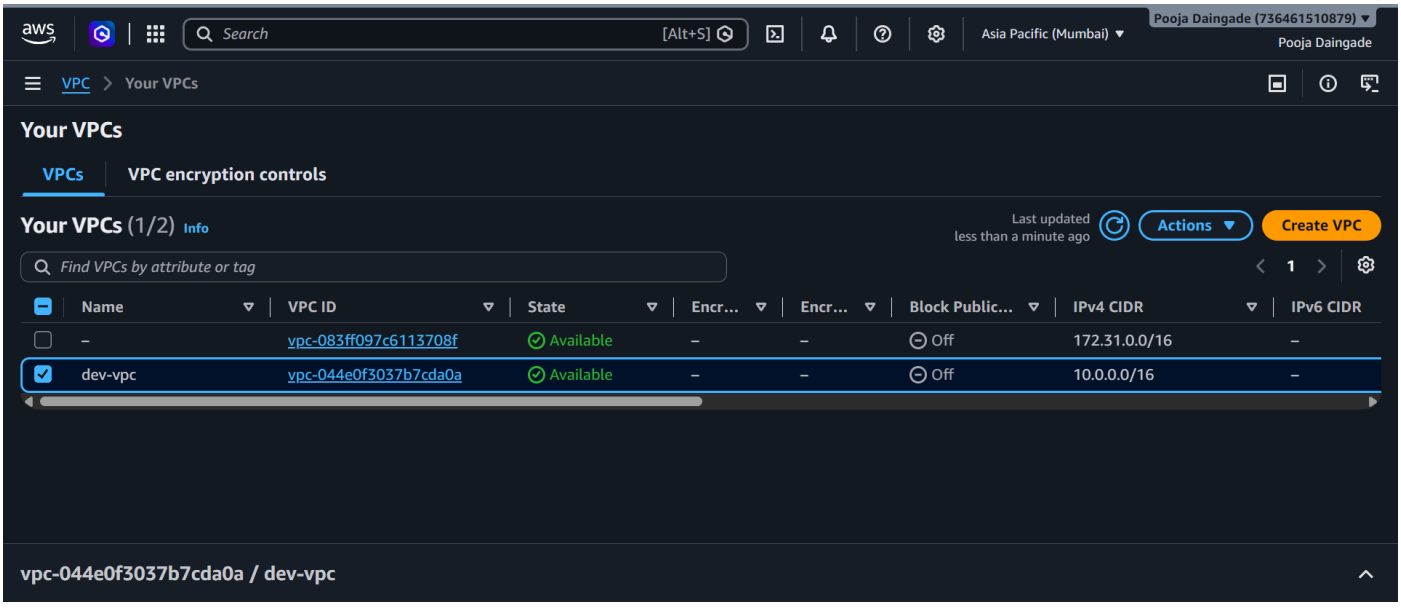
   The VPC was created using Terraform to provide an isolated network environment for the project.

   Configuration Details:

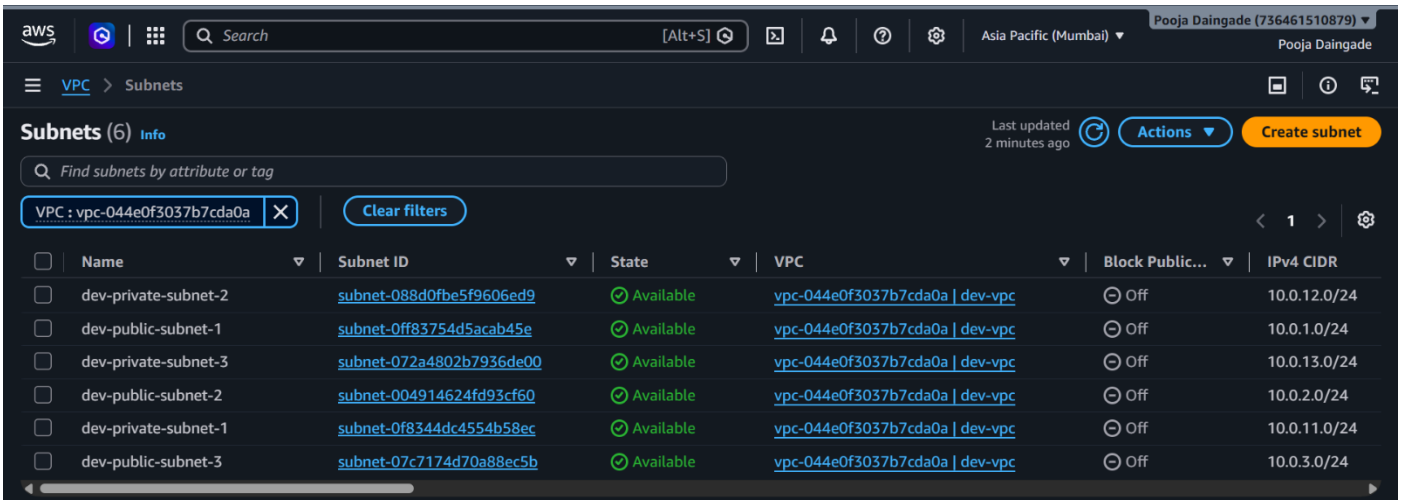
   - VPC Name: dev-vpc
   - VPC CIDR: 10.0.0.0/16
   - AWS Region: ap-south-1 (Mumbai)
   - Terraform was used to provision the VPC resources.



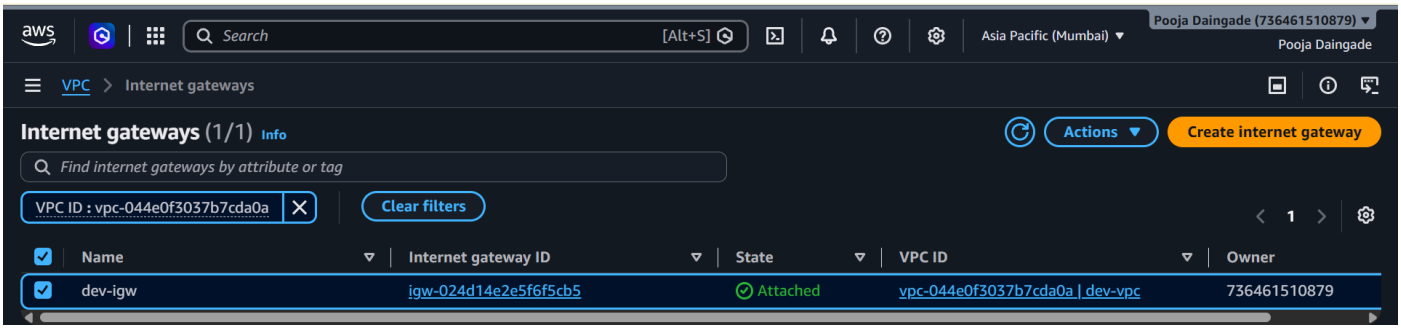
   - The VPC was successfully created and verified in the AWS Management Console.



- Six subnets were created across three Availability Zones to separate public-facing and private resources.
  CIDR Configuration:
- Public subnets: 10.0.1.0/24, 10.0.2.0/24, 10.0.3.0/24
- Private subnets: 10.0.11.0/24, 10.0.12.0/24, 10.0.13.0/24



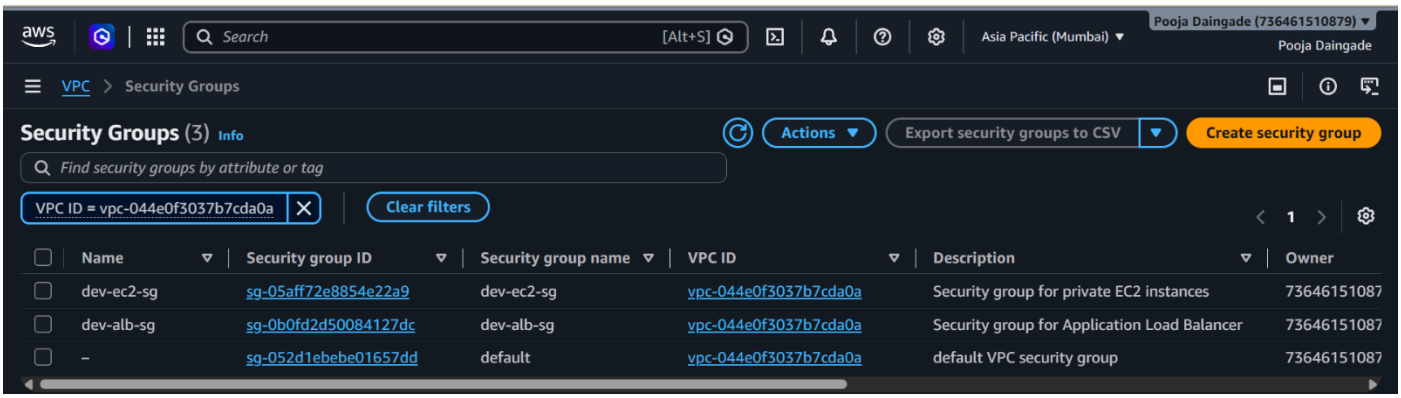
- An Internet Gateway was created and attached to the VPC to provide internet connectivity to resources in public subnets through the appropriate routing configuration.

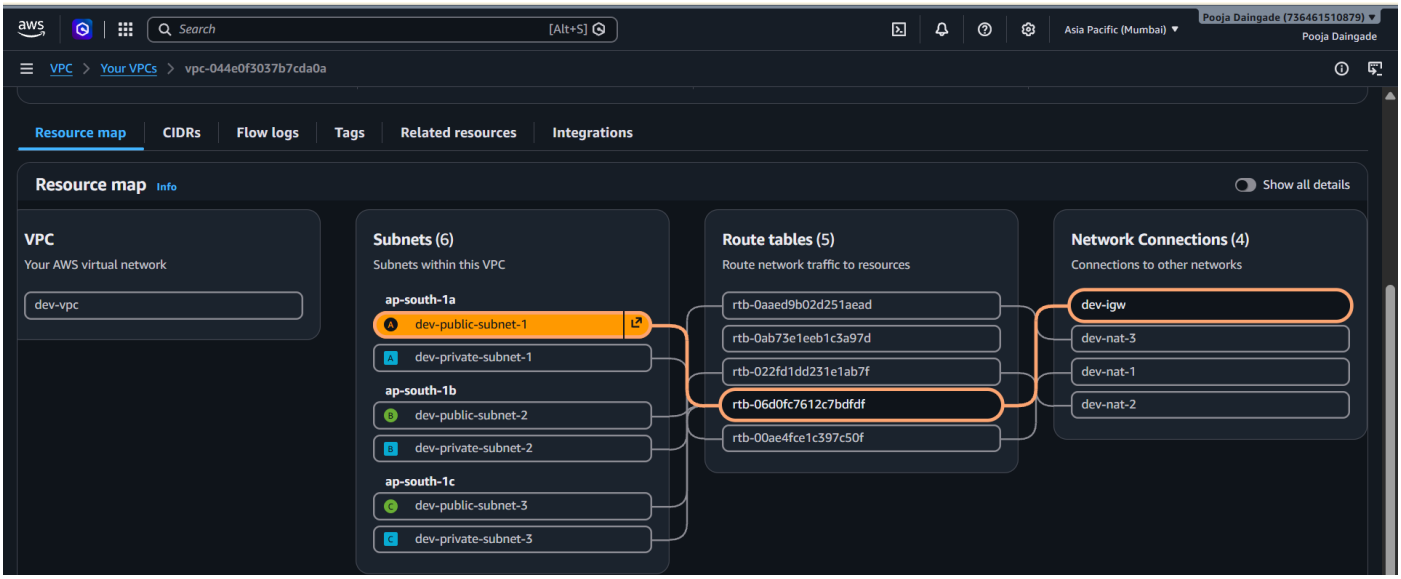


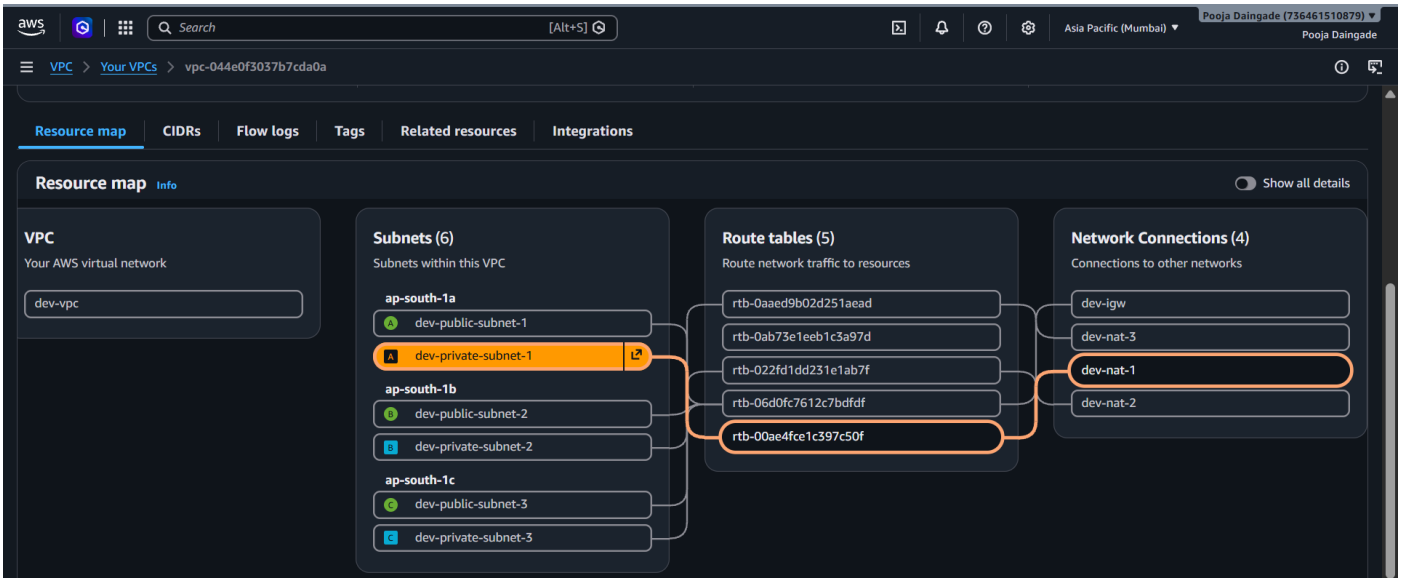
- Three NAT Gateways were created to support outbound internet access from resources in private subnets.



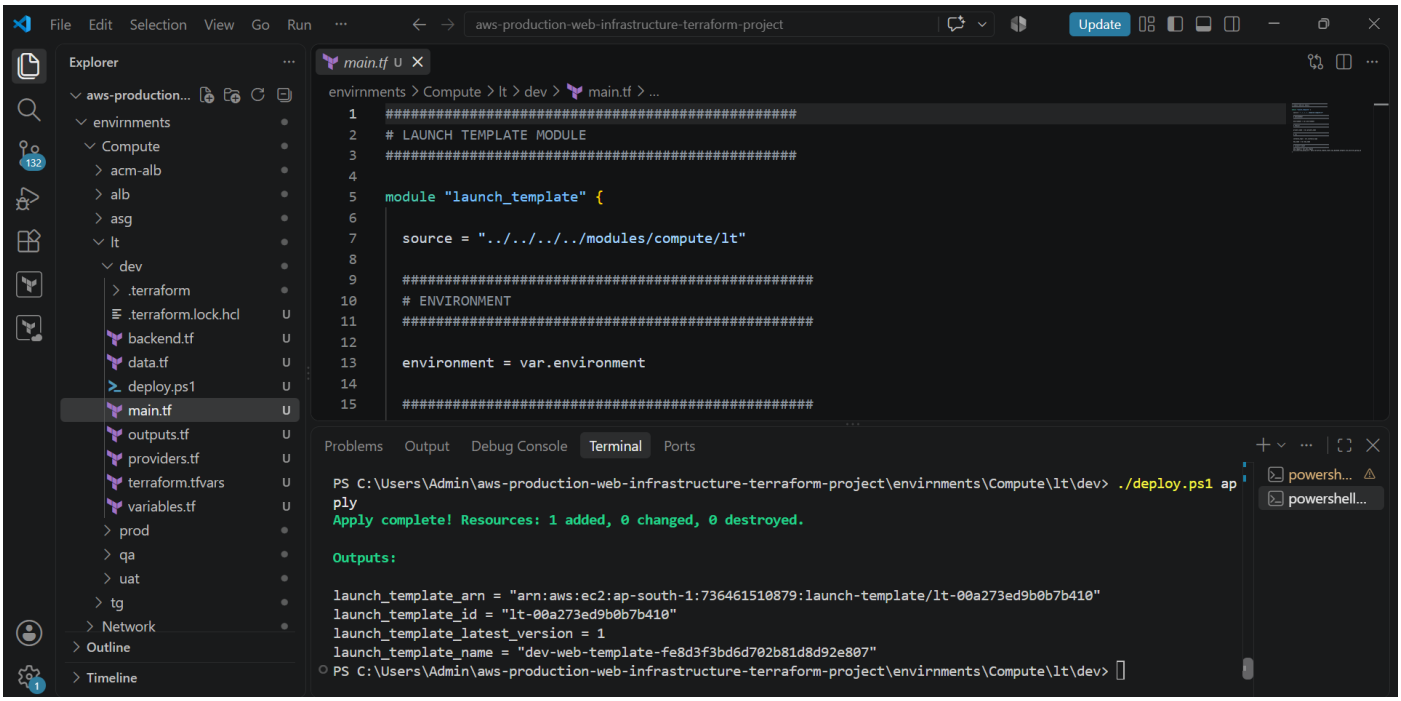
- Security groups were created to control network traffic for the Application Load Balancer and private EC2 instances.
  Security Groups:
  dev-alb-sg — Security group for the Application Load Balancer.
  dev-ec2-sg — Security group for private EC2 instances.
  These security groups help control inbound and outbound traffic according to the configured rul.



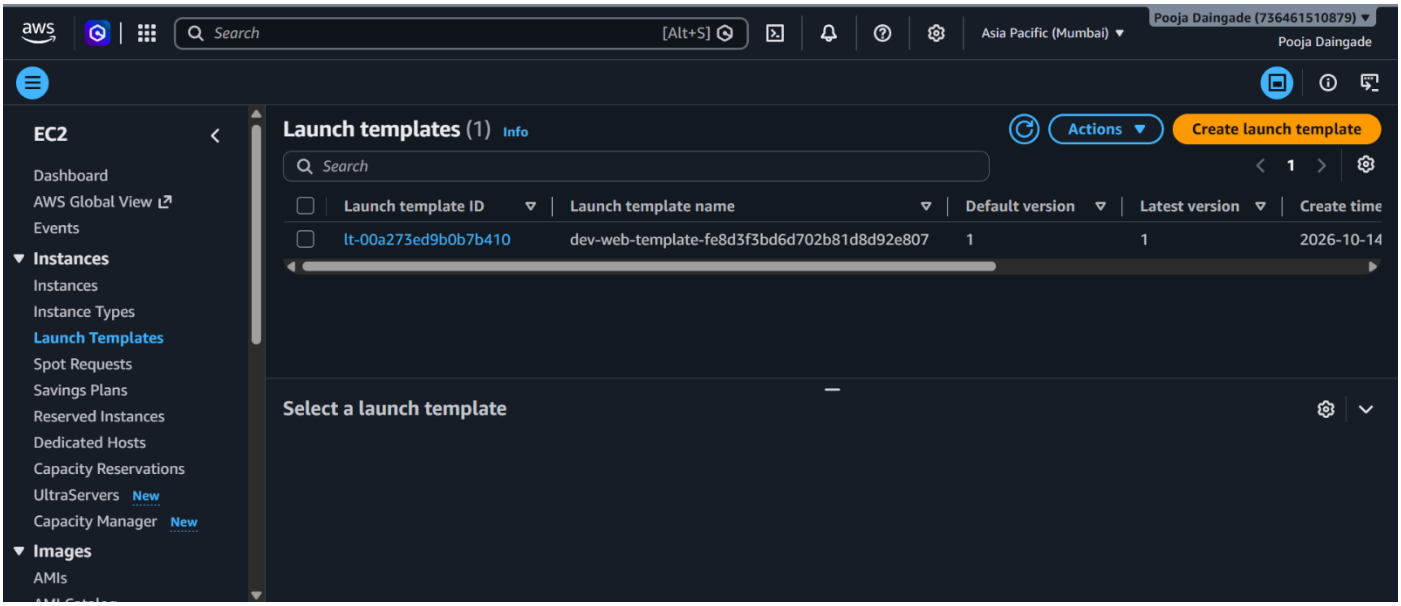




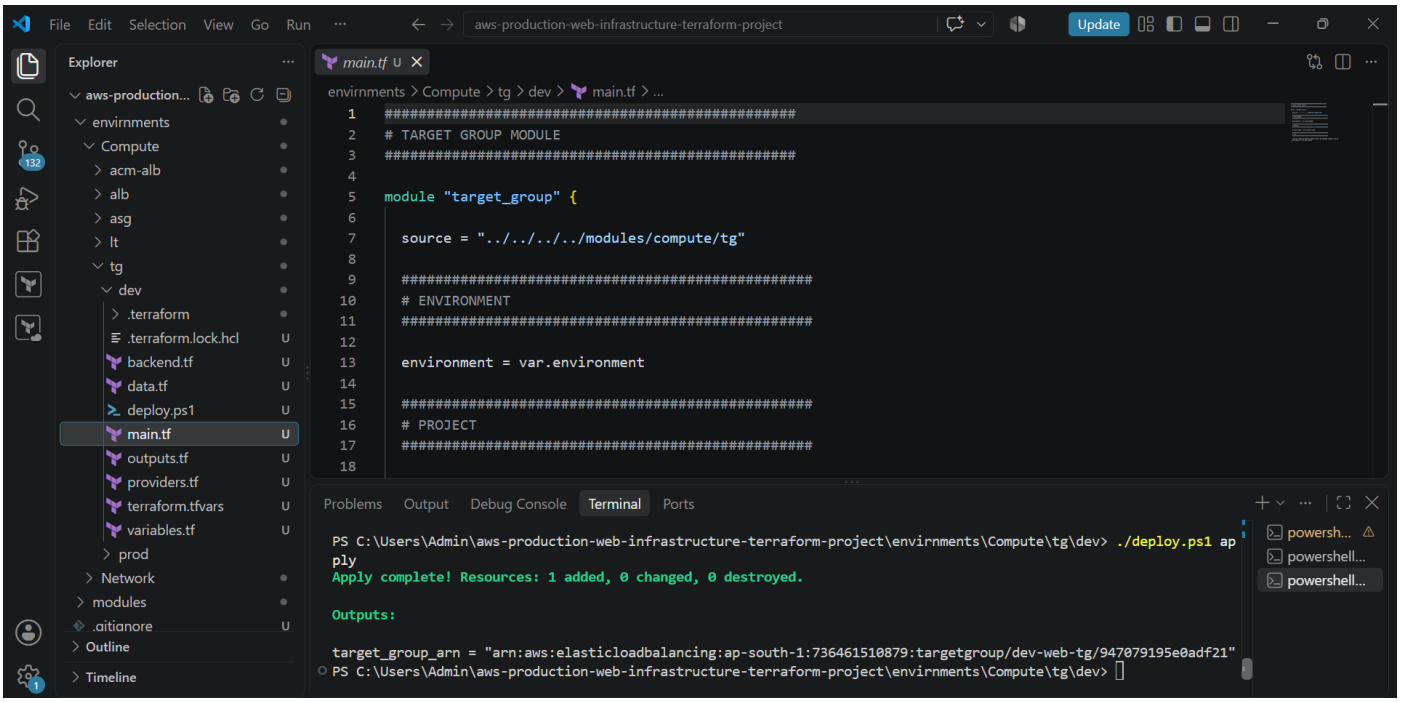
2. The Launch Template was configured using Terraform in VS Code. The Terraform configuration was applied successfully, creating one resource.



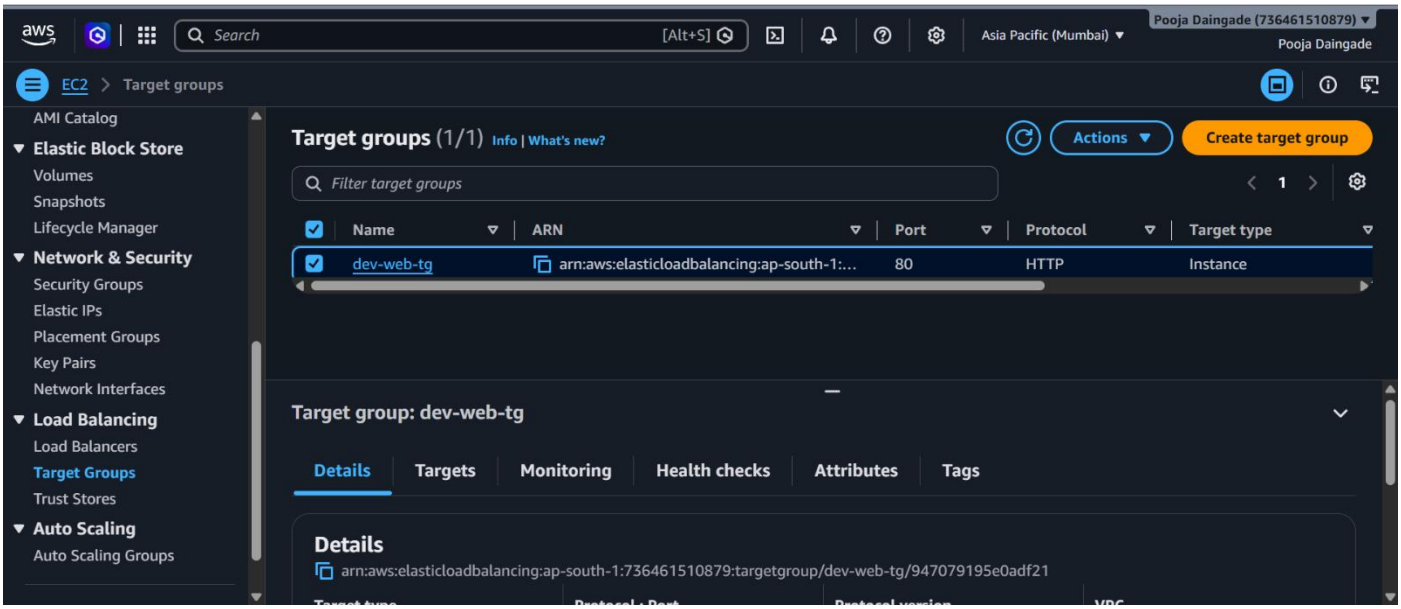
- The created Launch Template was verified in the AWS Management Console. Launch Template Name: dev-web-template-fe8d3f3bd6d702b81d8d92e807



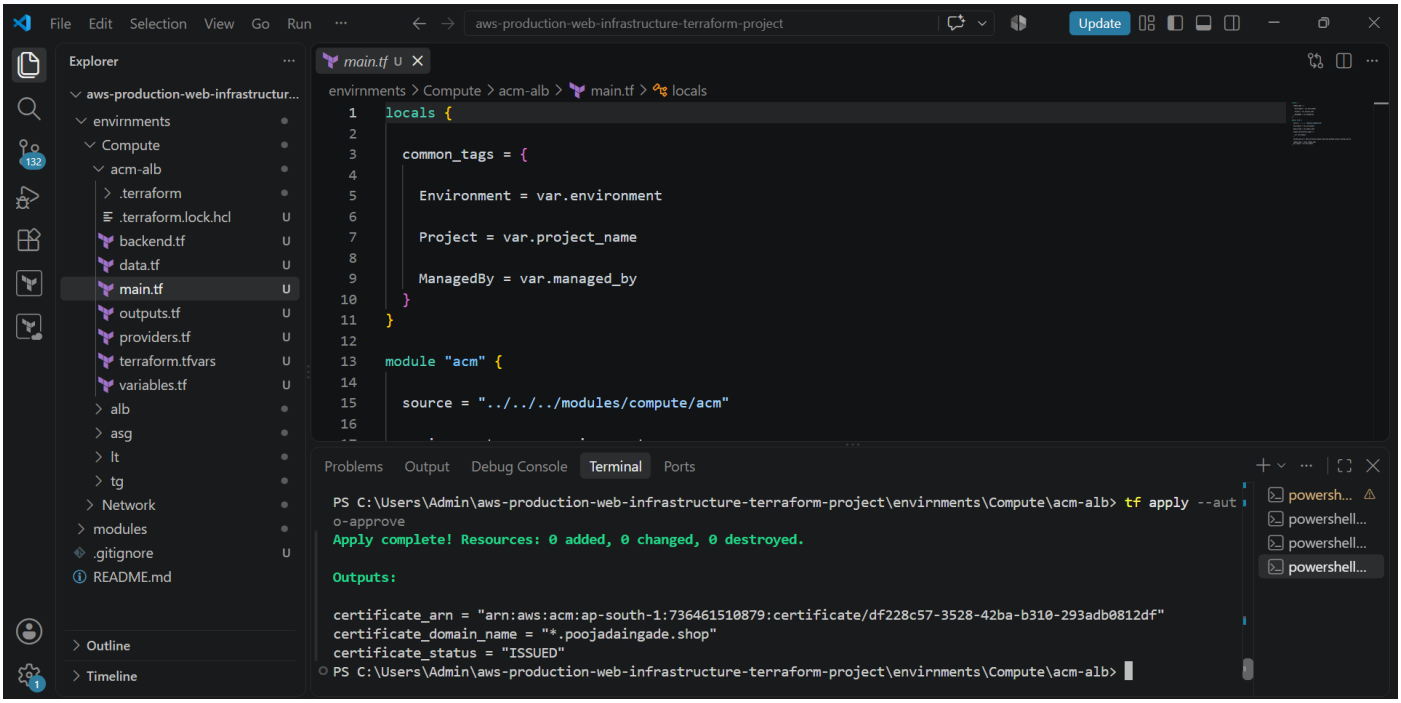
3. The Target Group was created using Terraform to register EC2 instances and enable health checks for the application traffic.



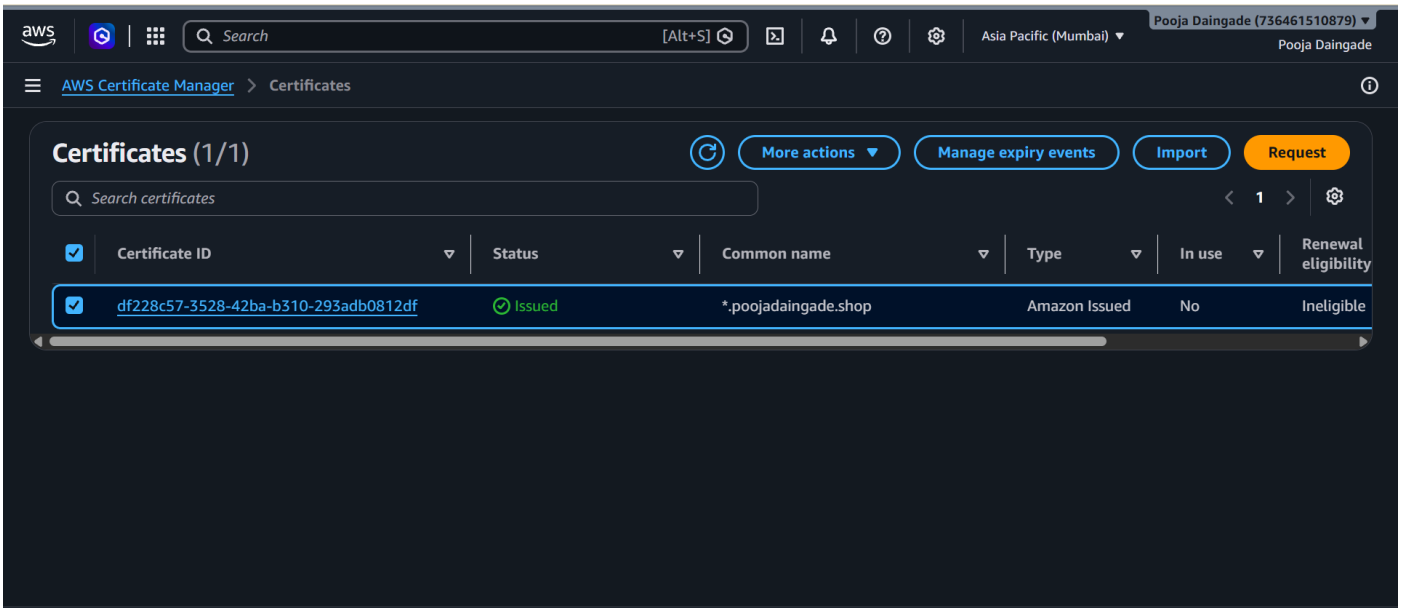
- The Target Group was verified in the AWS Management Console.
  Target Group Name: dev-web-tg
  Target Type: Instance
  Protocol: HTTP
  Port: 80



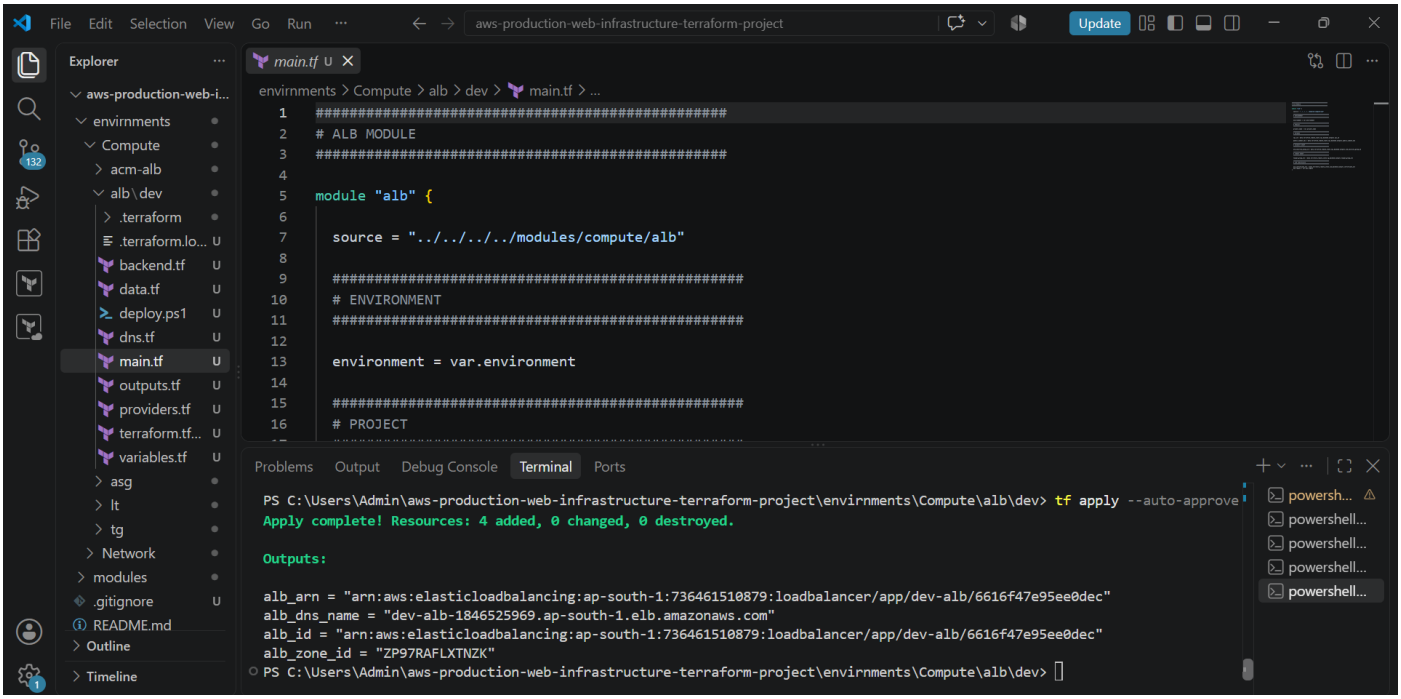
4. Created an ACM certificate using a reusable Terraform module.



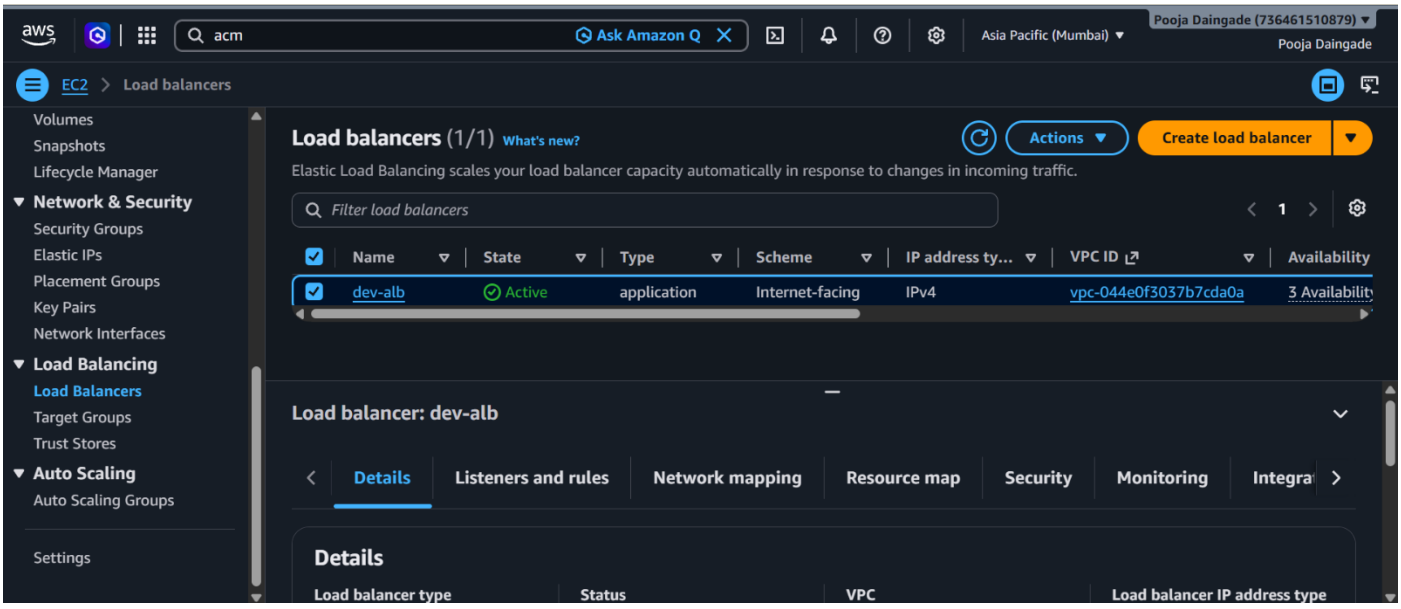
- Verified the ACM certificate in the AWS Console.
  Domain Name: *.poojadaingade.shop
  Certificate Status: ISSUED
  Certificate Type: Amazon Issued
  AWS Region: Mumbai (ap-south-1)



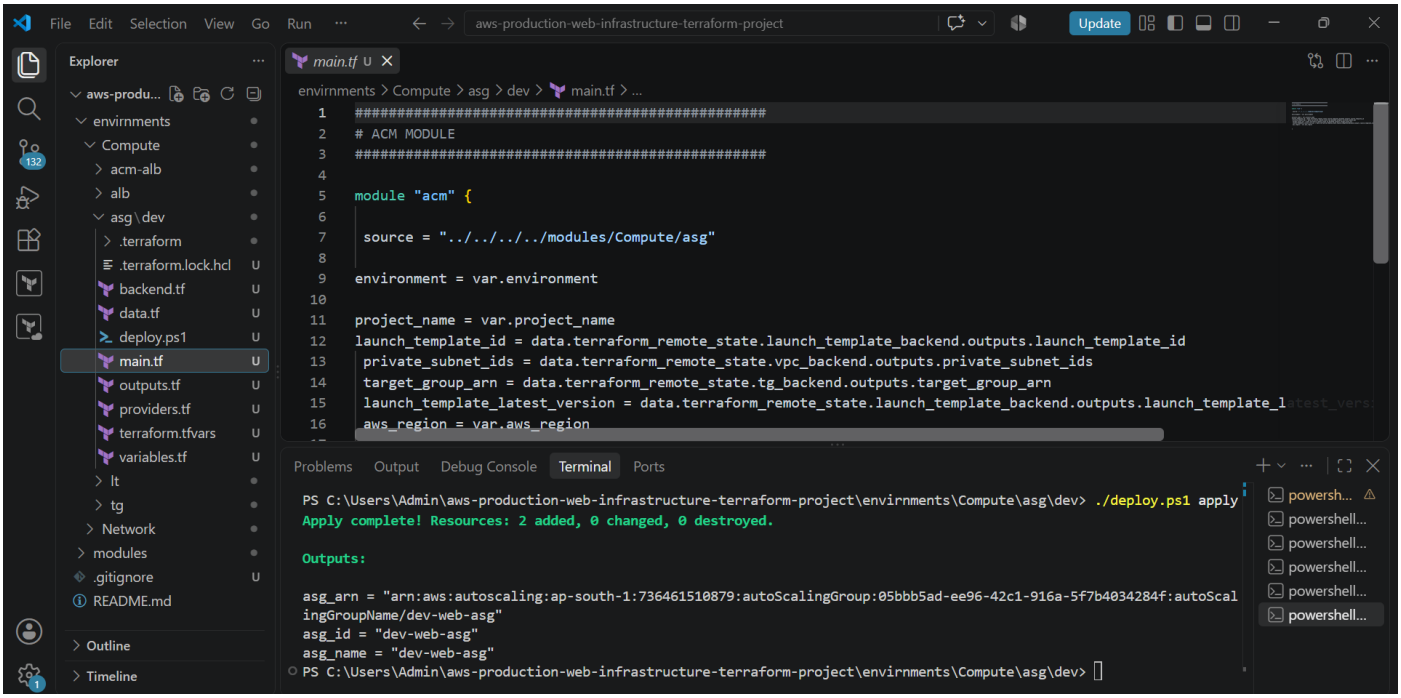
5. Created an AWS Application Load Balancer (**dev-alb**) using Terraform.



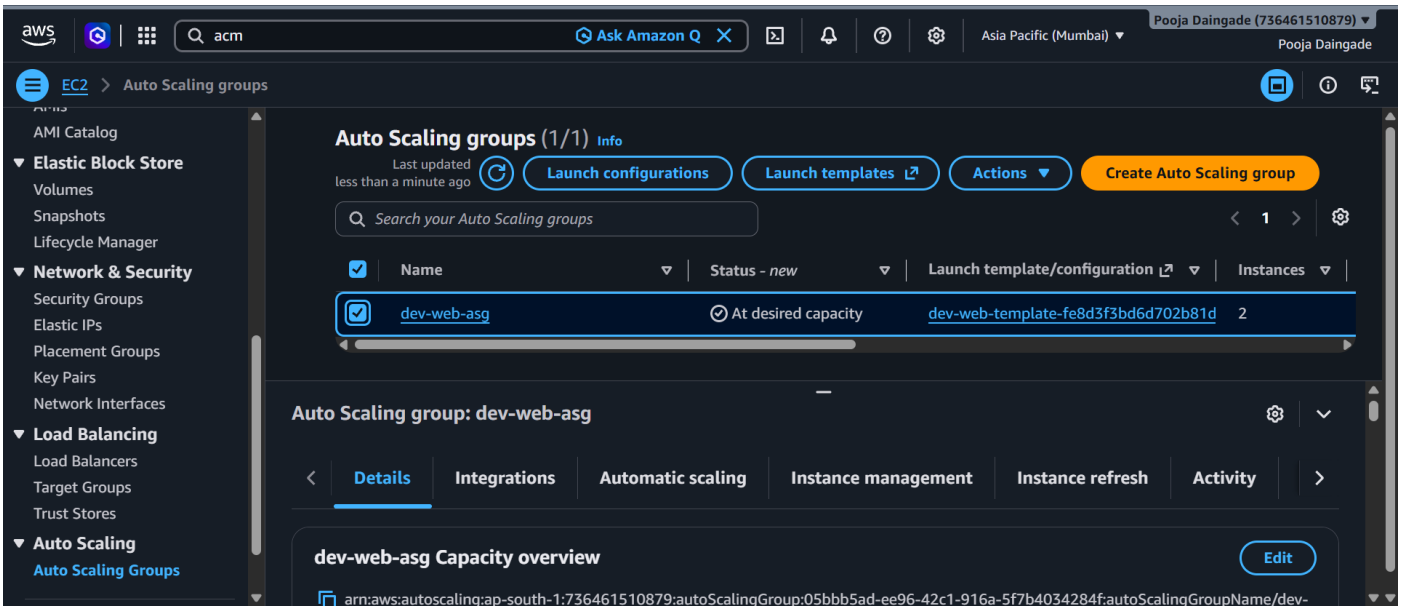
- Verified the Application Load Balancer named **dev-alb** in the AWS Management Console.



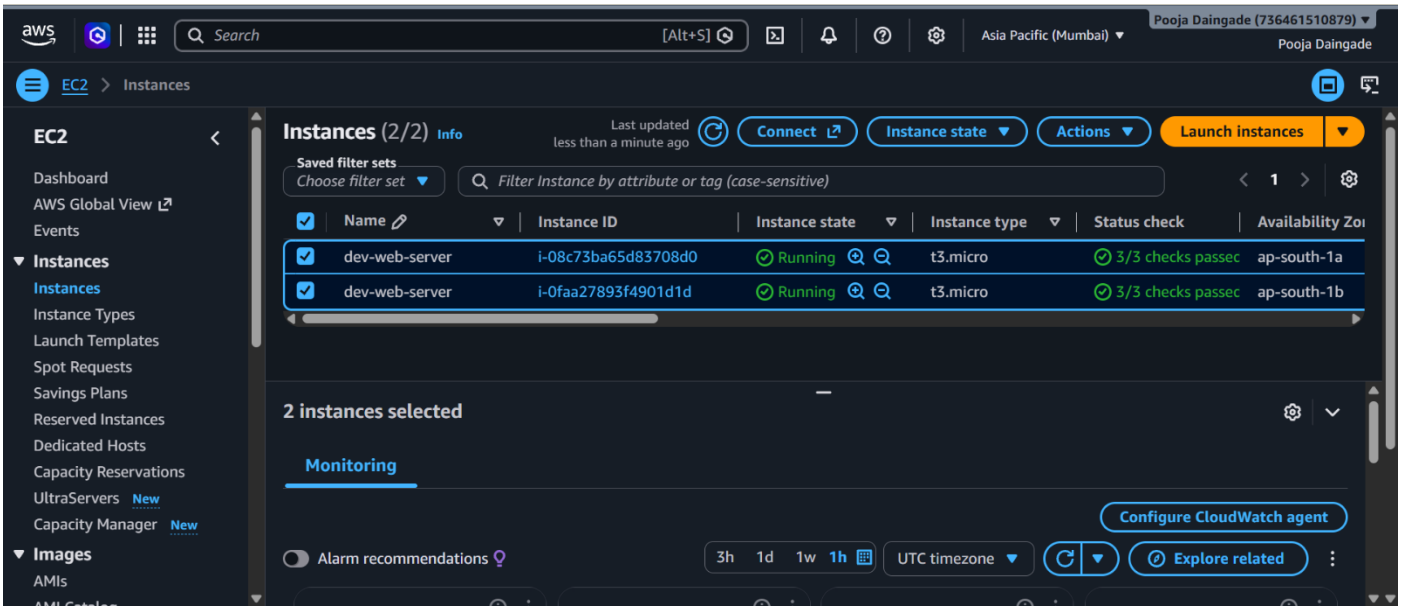
6. Created an Auto Scaling Group named **dev-web-asg** using Terraform.



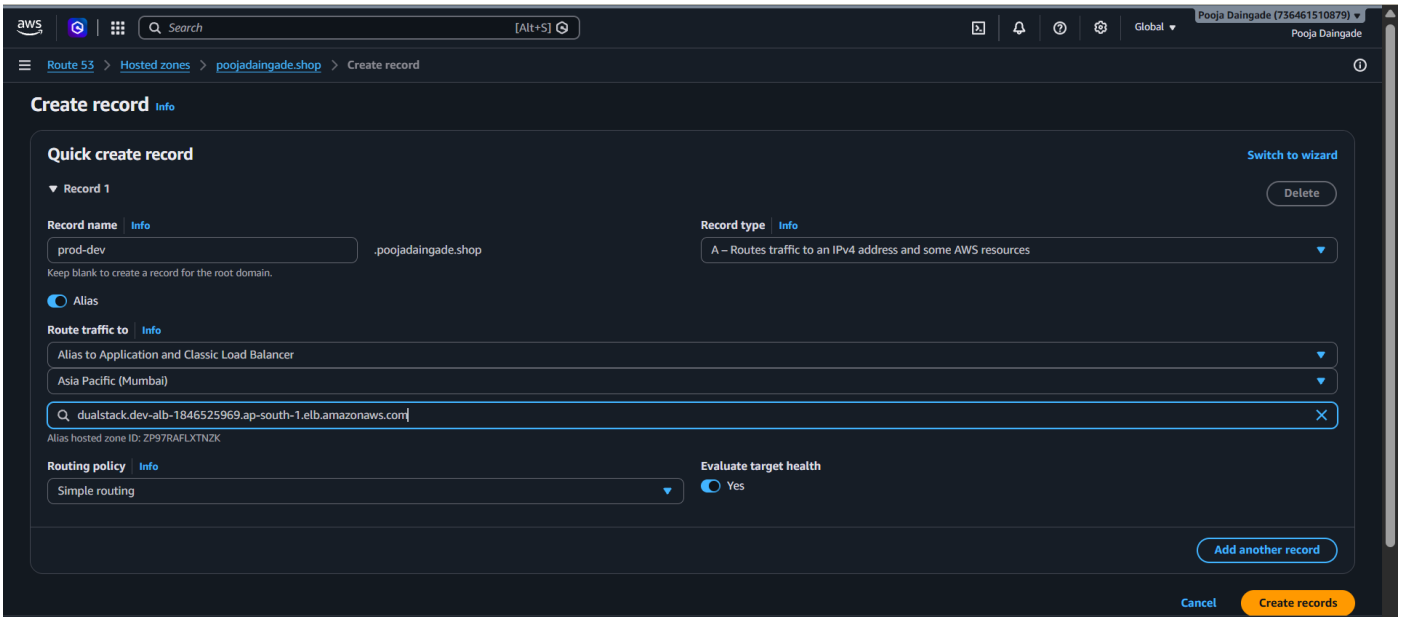
- Verified the Auto Scaling Group **dev-web-asg** in the AWS Management Console.



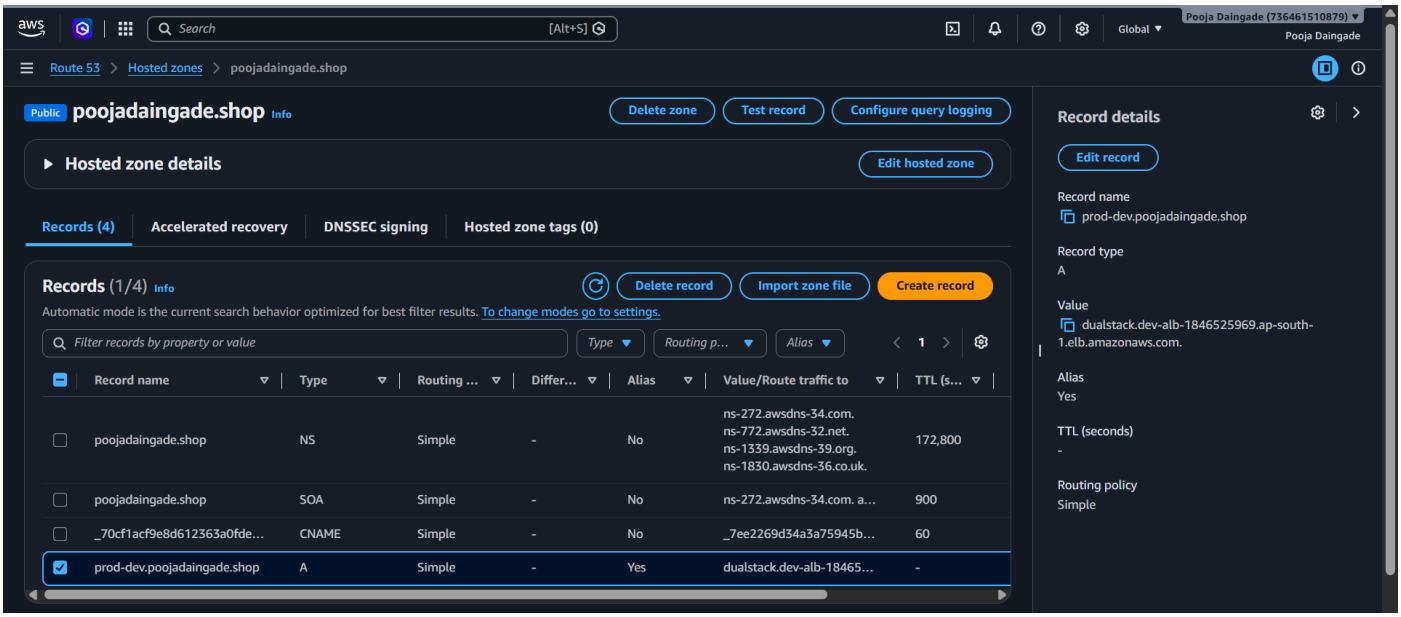
7. The Auto Scaling Group (dev-web-asg) used the configured Launch Template to automatically launch two EC2 instances named **dev-web-server**.



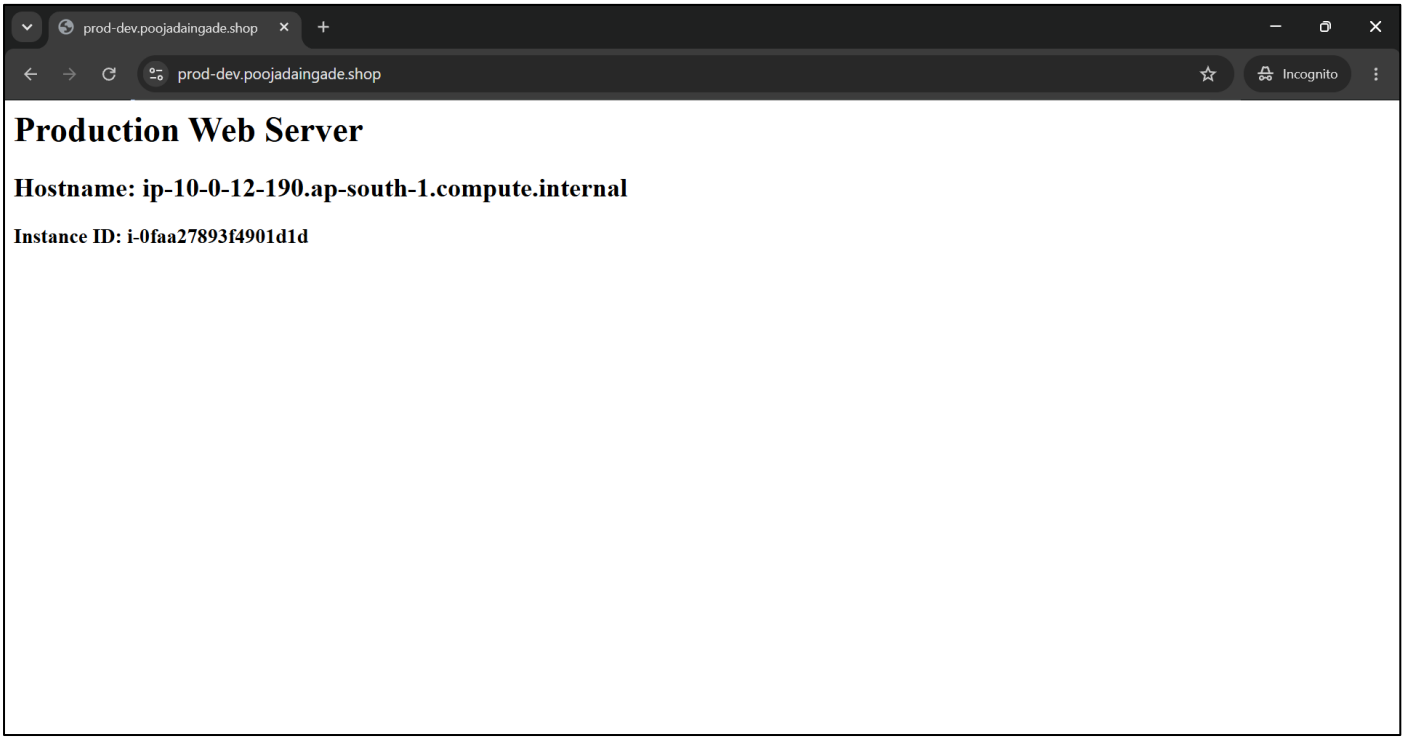
8. Created an A record for **prod-dev.poojadaingade.shop** in Amazon Route 53 and enabled Alias to route traffic to the Application Load Balancer in the Mumbai region.

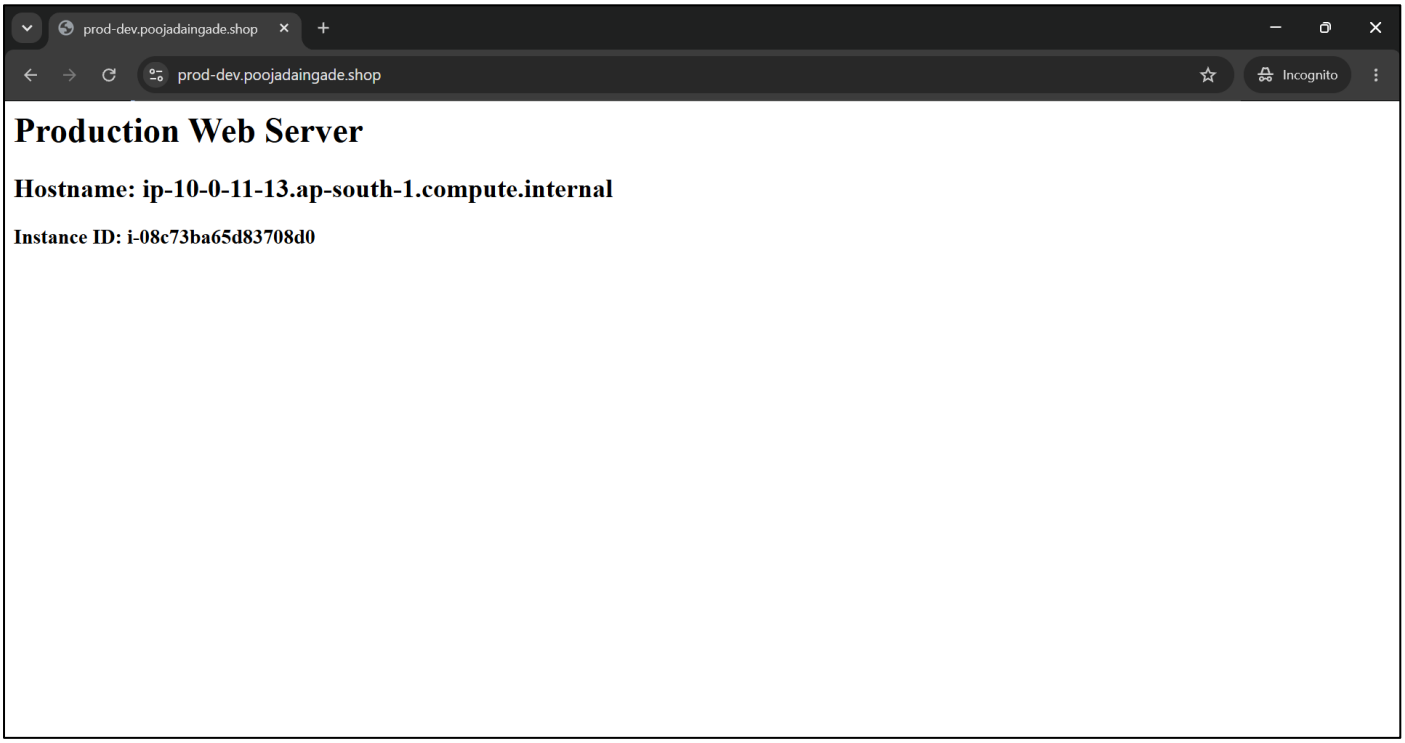


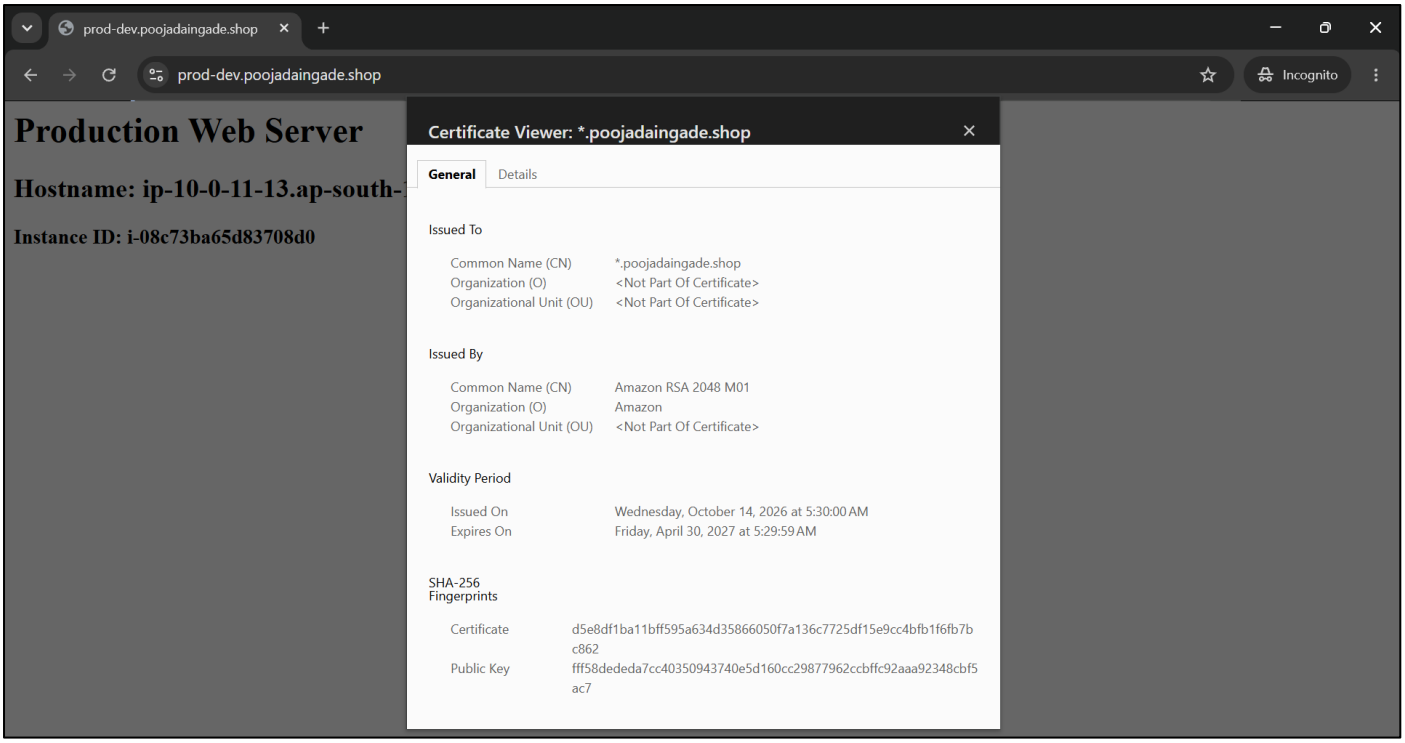
- Successfully created and verified the A record for **prod-dev.poojadaingade.shop**, routing domain traffic to the selected Application Load Balancer.



9. Successfully accessed **prod-dev.poojadaingade.shop** and verified traffic distribution between two EC2 instances using ALB and Auto Scaling Group.







Conclusion

Successfully implemented a production-ready AWS web infrastructure using Terraform. Configured VPC, public and private subnets, NAT Gateways, EC2, Application Load Balancer, Auto Scaling Group, ACM SSL/TLS certificate, and Route 53.

Verified secure HTTPS access through the custom domain and tested traffic distribution between EC2 instances. This project provided hands-on experience with AWS services and Infrastructure as Code (IaC).